Dynamic and Static WebSocket options in the Voice Streaming page

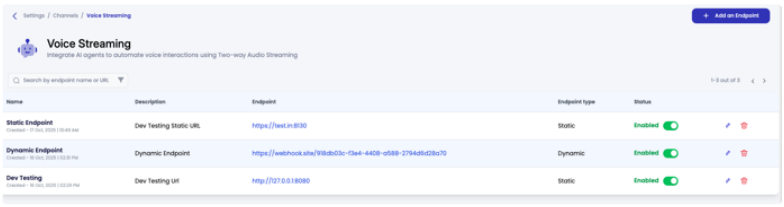
- **Author**: Gaurav Sharma
- **Project/Module**: VPaaS-Streaming

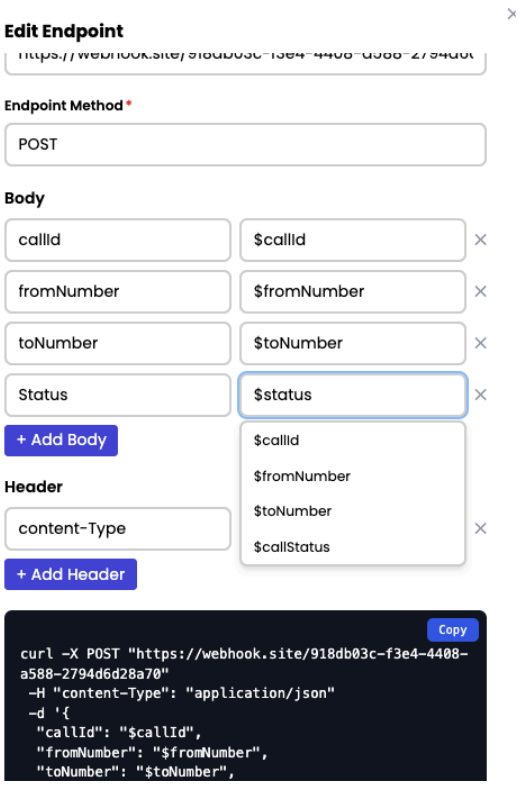
> **Purpose**
> This document explains how to configure **Static** and **Dynamic** endpoints for two-way audio streaming on the **Voice Streaming** page. It covers the new *Endpoint type* field, request/response contracts, timeouts, live request preview, and gives ready-to-run examples (NestJS & Express) for a Dynamic endpoint that returns a WebSocket URL.

What's new

- **Endpoint type** — choose between:
  - **Static**: a direct `wss://` URL for your voice bot.
  - **Dynamic**: a `POST` / `GET` HTTP endpoint that returns the `wss://` URL at call time.
- **Predefined variables** you can inject into the Dynamic request:
  - `$callId`, `$fromNumber`, `$toNumber`, `$status`
- **Custom variables** — add any additional fields you need; they will be sent with the request.
- **Real-time request preview** — see the exact cURL request the platform will send; copy/paste to test your endpoint quickly.
- **Strict response contract** for Dynamic endpoints (detailed below).
- **Timeouts/Failure behavior** — strict SLA on Dynamic resolution.

UI reference:



- 

-

When to use which endpoint type

Use case	Choose	Why
You always connect to the same bot instance	**Static**	Fast & simple: the platform connects straight to a fixed `wss://…` URL.
You route calls dynamically (e.g., by `toNumber`, tenant, language, business hours, AB tests)	**Dynamic**	Your service computes and returns the right `wss://…` per call.

Add an Endpoint

1. **Go to**: `Settings → Channels → Voice Streaming`.
2. Click **Add an Endpoint**.

   By default, the *Endpoint type* is **Static URL**.
3. Fill **Name** and **Description**.
4. Choose **Endpoint type**:
   - **Static**: paste the direct `wss://` URL.
   - **Dynamic**: select `GET` or `POST`, configure **Body** and **Headers**.
5. Use the **request preview** to verify what will be sent.
6. Click **Save** and toggle **Enabled** to activate.

Static Endpoint

- Provide the **direct WebSocket URL** of your voice bot (e.g., `wss://bot.example.com/stream`).
- The platform connects to this URL and streams **events** and **media** over that socket.

Dynamic Endpoint

Your service is called in real time to return the WebSocket URL to use for this call.

Request (what we send to you)

- **Method**: `POST` (JSON body) or `GET` (query string). You choose in the UI.
- **Headers**: At minimum, set `content-type: application/json` for `POST`. You may add more headers (e.g., auth).
- **Body / Query parameters**: you map keys to variables in the UI.

Predefined variables

Variable	Meaning
`$callId`	Unique identifier for the call.
`$fromNumber`	Caller's number (E.164 if available).
`$toNumber`	Destination number.
`$status`	Call status at the moment of the request (string).

Custom variables

- Add any key/value pairs you need (e.g., `tenantId`, `locale`, `plan`).

- They are sent alongside the predefined variables.

Example `POST` request body

```
{
  "callId": "$callId",
  "fromNumber": "$fromNumber",
  "toNumber": "$toNumber",
  "status": "$status",
  "tenantId": "retail-us"
}

```

At runtime the variables are replaced with actual values.

Example cURL (from the preview)

```
curl -X POST "https://your-domain.example.com/voice/endpoint" \
  -H "content-type: application/json" \
  -d '{
    "callId": "c-3f9e17",
    "fromNumber": "+15551234567",
    "toNumber": "+15557654321",
    "status": "ringing",
    "tenantId": "retail-us"
  }'
```

(For `GET`, the same fields are sent as query parameters instead of JSON.)

Response (what you must return)

Your service **must** respond within **2000 ms** with **HTTP 200** and **exactly** the following JSON:

```
{
  "sucess": true,
  "wss_url": "wss://<your-bot-host>/stream?token=..."
}

```

> ⚠️ **Important**
>
> - The keys must match **exactly** as shown above (`sucess` and `wss_url`).
> - Any other keys, a non-200 status code, invalid JSON, or missing keys will cause the request to be **declined** and the call will **hang up**.
> - If your existing service uses a differently spelled flag (e.g., `success`), update it to match the required schema above.

JSON schema (reference)

```
{
  "type": "object",
  "properties": {
    "sucess": { "type": "boolean" },
    "wss_url": { "type": "string", "pattern": "^wss://.+" }
  },
  "required": ["sucess", "wss_url"],
  "additionalProperties": false
```

```
}
```

End-to-end flow (Dynamic)

1. Platform sends `GET` / `POST` to your Dynamic endpoint with mapped variables.
2. Your service computes the destination `wss://…` (optionally using custom variables).
3. Your service returns `200` with JSON `{ "sucess": true, "wss_url": "wss://…"}` within **<= 2000 ms**.
4. Platform connects to the returned `wss://…`.
5. All **events** and **audio media** are streamed over that WebSocket.

If any step fails (non-200, wrong payload, timeout > 2000 ms), the request is **declined** and the call **hangs up**.

Operational rules & limits

- **Response deadline**: **2000 ms**. Exceeding this results in **hang up**.
- **HTTP code**: must be **200**.
- **Response JSON**: must contain only the expected keys:
  - `sucess` (boolean)
  - `wss_url` (string, must be a valid `wss://` URL)
- **On failure** (wrong payload/code/timeout): request is declined and the call is **hung up**.
- After connection, **all events and audio media are transmitted via the returned `wss://` URL**.